

Former Knowns: A Belief Lifecycle Engine for Temporal Authority and Recovery in Long-Lived AI Agents

Eric Sawtelle

Independent Researcher
e.sawtelle358@gmail.com

Revised peer-review manuscript - June 2026

Abstract

Across 750 frozen validation runs, positive-evidence drift recovery reduced unnecessary valid-action rejection by 75.11% while preserving zero observed unsafe actions, zero false-invalidation events, and unchanged permanent-change gating latency. All four recoverable-scenario valid-rejection comparisons remained significant after Holm-corrected paired Wilcoxon signed-rank analysis, with paired bootstrap confidence intervals excluding zero. The candidate produced no unsafe action in 150 validation runs; the one-sided exact 95% upper bound on the run-level event probability was 1.98%. These results evaluate the Belief Lifecycle Engine (BLE), an auditable architecture for the *former-known* problem: propositions that were once correct and operationally justified but become invalid while retaining authority over action. BLE separates epistemic confidence from verification freshness, records contradiction as drift, represents temporal modes, quarantines anomalies through a SUSPECT state, propagates concern according to typed dependency semantics, and gates actions by belief state and consequence risk. A staged simulator program exposed false invalidation, collateral dependency blocking, and sticky recovery before the frozen validation study. Five systems were compared across five scenarios and 30 previously unused seeds per condition. Static Memory and Fixed TTL served as sanity-check baselines; BLE Authority was the control; fixed-rate drift recovery and adaptive recovery were candidate policies. Fixed-rate drift recovery was retained because it matched or exceeded adaptive recovery while remaining simpler. The findings support an integrative systems claim: long-lived agents require explicit temporal authority management, and restored evidence should actively reduce accumulated distrust.

Keywords: long-term AI memory; belief revision; temporal authority; truth maintenance; concept drift; persistent agents; epistemic hysteresis; memory safety

Contents

1. Introduction
 2. Former Knowns as an Epistemic and Operational Category
 3. Related Work
 4. Belief Lifecycle Engine
 5. Research Questions
 6. Experimental Method
 7. Results
 8. Discussion
 9. Limitations and Threats to Validity
 10. Conclusion
- References
- Appendix A. Formal State and Transition Summary
- Appendix B. Complete All-System Results
- Appendix C. Frozen Parameter Configuration
- Appendix D. Reproducibility Checklist

1. Introduction

Persistent AI agents are moving from single-session tools toward systems expected to operate across days, months, and changing environments. Their memory modules may store user preferences, API contracts, credentials, workflow dependencies, organizational rules, device states, and summaries of prior interactions. The prevailing engineering question has been how to extend context, select memories, consolidate experience, and retrieve relevant information. That work is necessary, but it leaves a distinct operational problem insufficiently specified: a memory can remain internally coherent, retrievable, and once well-supported while no longer describing the present world.

We call such information a *former known*. A former known is not merely an initial error, an unresolved unknown, or a hallucinated claim. It is a proposition that satisfied an agent's acceptance criteria at an earlier time and then silently lost validity. Examples include an endpoint that accepted version-two tokens before a deployment, a user preference later reversed, a device that changed ownership, a scheduled process that stopped running, or a dependency that was removed. Historical correctness gives such memories unusual persuasive force. They are often retrieved precisely because they were repeatedly useful, and their provenance may appear stronger than a newly observed contradiction.

Classical truth-maintenance systems track justifications and retract consequences when support changes [1,2]. AGM belief revision formalizes rational contraction and revision [3], while knowledge-base update distinguishes new information about a changed world from correction of a mistaken description [4]. Temporal databases and knowledge graphs represent validity through time [5,14,15], and concept-drift research studies adaptation under changing data-generating processes [6]. Continual learning, by contrast, has focused heavily on preventing unwanted loss of prior capabilities [7,8]. Recent LLM-agent memory research emphasizes extended context, retrieval, reflection, and selectively preserved experience [9-13]. These traditions provide essential building blocks, but they do not by themselves specify an operational authority system for long-lived agents in which confidence, freshness, dependency semantics, consequence risk, and recovery are jointly managed.

This paper presents the Belief Lifecycle Engine (BLE), a systems architecture for that purpose. BLE treats a belief as a versioned operational object rather than a timeless string. Its design is guided by five principles:

1. Confidence and freshness are distinct. Lack of recent verification does not itself prove falsehood.
2. Temporal behavior is domain-specific. Continuous, event-bound, epoch-bound, and static beliefs should not age identically.
3. Contradiction and dependency concern require provenance. A locally contradicted belief differs from one made temporarily suspect by a correlated upstream failure.
4. Authority is action- and risk-dependent. A stale belief may remain usable for a reversible read while being insufficient for a critical write.
5. Recovery must be controlled but possible. Positive evidence should actively reduce accumulated distrust after transient failure or observation noise.

The paper makes four contributions. First, it formalizes the former-known problem as a temporal authority failure distinct from ordinary factual error and catastrophic forgetting. Second, it specifies BLE as an auditable integration of confidence, freshness, drift, lifecycle states, typed dependency propagation, and risk-sensitive action gating. Third, it reports a staged experimental program in which progressively identified failure modes were corrected rather than hidden. Fourth, it presents a frozen 750-run validation study showing that positive-evidence drift recovery substantially reduces unnecessary action rejection without degrading the tested safety or persistent-change response.

The claims are intentionally bounded. No individual BLE mechanism is asserted to be wholly unprecedented. The contribution is the operational integration, the explicit distinction between remembered truth and present authority, and a falsifiable evaluation showing how design choices affect safety, availability, and recovery in a controlled long-lived-agent setting.

2. Former Knowns as an Epistemic and Operational Category

2.1 Definition

Let ϕ be a proposition used by an agent at times t_0 and t_1 , where $t_1 > t_0$. A proposition is a former known at t_1 when: (i) ϕ was true in the environment at t_0 ; (ii) the agent possessed sufficient evidence to treat ϕ as operationally trusted at t_0 ; (iii) ϕ is false at t_1 ; and (iv) the agent's current memory state still grants ϕ enough authority to influence action. The critical property is not historical falsity but *authority persistence after validity loss*.

This category clarifies a practical difference between several failure modes. A hallucination was never adequately grounded. A known unknown is explicitly unresolved. Catastrophic forgetting removes useful prior knowledge during

learning [7,8]. A former known, by contrast, is retained too successfully: it remains available and persuasive after the world has changed.

2.2 Revision, update, and expiration

The former-known problem contains elements of both revision and update. A contradictory observation may reveal that the original belief was wrong, which resembles revision, or that the world has changed, which resembles knowledge-base update [3,4]. A persistent agent often cannot immediately distinguish those cases. BLE therefore delays metaphysical classification and instead manages operational authority: it records evidence, reduces or restores authorization, schedules verification, and preserves audit history.

Expiration is also not uniform. Mathematical identities are effectively static within the operational domain. User preferences can change irregularly. Service contracts may change abruptly. Heartbeats and backups are event-bound: their meaning depends on whether an expected event occurred, not on smooth time decay. Treating every belief with a single forgetting curve creates category errors.

2.3 Authority rather than binary truth

BLE does not claim to solve objective philosophical truth. It models whether an agent is justified in using a proposition for a particular action under present evidence and risk. This distinction is essential because the same belief can support one action but not another. A stale endpoint description might be adequate for displaying cached documentation, inadequate for sending a payment, and useful as a hypothesis for a diagnostic probe.

3. Related Work

3.1 Truth maintenance and belief revision

Doyle's truth-maintenance system represented beliefs with justifications and maintained a consistent set of accepted propositions as assumptions changed [1]. De Kleer's ATMS extended this approach by tracking environments of assumptions, enabling efficient reasoning across alternative contexts [2]. BLE inherits the importance of explicit dependencies and justification history but adds graded confidence, temporal freshness, drift accumulation, typed propagation, action-risk policy, and recovery dynamics.

AGM theory provides foundational postulates for contraction and revision [3], while later work distinguishes revision of a mistaken knowledge state from update after a changing world [4]. BLE is less general as a logic but more operational as an agent-control system. It does not attempt a representation theorem. Instead, it defines concrete state variables and transition rules that can be executed, logged, and empirically tested.

3.2 Temporal information and concept drift

Temporal database research distinguishes valid time from transaction time and provides structures for historical states [5]. Temporal knowledge-graph work similarly associates facts and rules with timestamps or validity intervals [14,15]. Recent work explicitly argues that confidence is not timeless and learns rule-specific decay behavior [14]. BLE shares the premise that temporal validity matters but separates freshness from confidence to avoid treating elapsed time as direct negative evidence.

Concept-drift research studies changes in the statistical relationship between observations and targets [6]. BLE operates at a more granular symbolic-control level: individual beliefs can become obsolete, observations can be noisy, and downstream actions can carry heterogeneous consequences. The simulator's abrupt-change and transient-outage scenarios are related to drift benchmarks, but BLE's dependent belief graph and action authority are central outcome variables.

3.3 Continual learning and forgetting

Catastrophic interference describes the rapid loss of prior knowledge when connectionist systems learn sequentially [7]. Continual-learning research addresses the stability-plasticity trade-off and seeks to preserve useful capabilities while adapting [8]. Former knowns expose the inverse risk: excessive stability can preserve operational authority for knowledge that should no longer be trusted. BLE therefore treats forgetting as neither inherently good nor bad. It distinguishes passive freshness loss, contradiction-driven confidence change, retirement, and reversible quarantine.

3.4 Memory in LLM-based agents

Generative Agents store observations, form reflections, and retrieve memories to guide planning [9]. MemGPT introduces hierarchical context management inspired by operating systems [10]. MemoryBank incorporates long-term

storage and time-based forgetting [11], while LongMem augments language models with external long-term memory [12]. Surveys of LLM-agent memory emphasize writing, storage, retrieval, summarization, and evaluation [13]. These systems demonstrate the value of persistence, but increased memory capacity amplifies the need for validity and authority controls. BLE is complementary: it can govern structured claims stored by an LLM agent regardless of the underlying retrieval substrate.

3.5 Research gap

Across these literatures, temporal metadata, dependency maintenance, decay, revision, and long-term storage are well established. The remaining gap is an integrated and experimentally specified answer to the following question: *When should a persistent agent stop using a remembered proposition to authorize action, and how should it regain authority after transient contradiction?* BLE is designed around that operational question.

4. Belief Lifecycle Engine

Belief Lifecycle Engine: evidence-to-action control loop

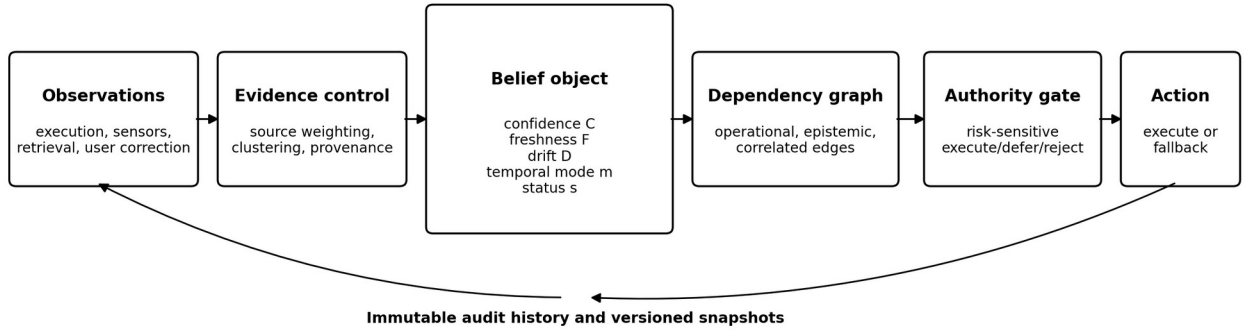


Figure 1. Belief Lifecycle Engine evidence-to-action control loop.

4.1 Belief object

Table 1. Canonical BLE belief-object fields.

Field	Meaning	Operational role
φ	Claim or proposition	Content evaluated against observations or execution
C	Confidence in [Cmin, Cmax]	Evidence-based support; not passively decayed
F	Freshness in [0,1]	Recency of successful verification
D	Drift score	Accumulated contradiction minus recovery
m	Temporal mode	CONTINUOUS, EVENT, EPOCH, or STATIC
s	Lifecycle state	TRUSTED, STALE, SUSPECT, RECONCILIATION, or RETIRED
Θ	Threshold bundle	Stop, recovery, stale, and risk-specific limits
R	Consequence risk	Expected impact if the claim is wrong
K	Validation cost	Cost of direct reality contact
G / P / H	Dependencies, provenance, history	Propagation meaning, suspicion origin, and immutable audit trail

A BLE belief is a versioned object:

$$b_i = \langle id_i, \varphi_i, C_i, F_i, D_i, m_i, s_i, \Theta_i, R_i, K_i, G_i, P_i, H_i \rangle \quad (1)$$

where id is a stable identifier; ϕ is the claim; C is epistemic confidence; F is verification freshness; D is accumulated drift; m is temporal mode; s is lifecycle status; Θ contains thresholds; R is consequence risk if wrong; K is validation cost; G is the dependency map; P is suspicion provenance; and H is immutable audit history. This structure is deliberately richer than a memory string because operational authority depends on how, when, and for what purpose a claim is used.

4.2 Confidence and freshness

Confidence $C \in [C_{min}, C_{max}]$ summarizes evidential support. Freshness $F \in [0,1]$ summarizes recency of verification. Passive time changes F , not C . For a continuously aging belief:

$$F_i(t) = \max(F_{min,i}, \exp[-\lambda_i (t - \tau_i)]) \quad (2)$$

where τ_i is the last verification time and λ_i is a belief-specific freshness rate. This separation was a central correction from the initial simulator: directly decaying confidence caused valid but unobserved beliefs to be falsely invalidated.

4.3 Temporal modes

Table 2. Temporal modes and aging semantics.

Mode	Aging rule	Example
CONTINUOUS	Freshness decays between observations	Service availability or network state
EVENT	Penalty begins after expected event plus grace	Nightly backup or heartbeat
EPOCH	Valid within a version; revalidate at boundary	API contract or credential generation
STATIC	No passive aging in modeled domain	Mathematical identity or fixed definition

BLE defines four temporal modes.

- CONTINUOUS beliefs gradually lose freshness between observations.
- EVENT beliefs are evaluated relative to an expected event interval and grace window.
- EPOCH beliefs remain valid within a version or epoch and require revalidation after a boundary.
- STATIC beliefs do not passively age within the modeled domain.

Temporal modes prevent event-bound claims such as "the nightly backup completed" from being forced into a smooth decay function and prevent mathematical or definitional facts from receiving arbitrary expiration.

4.4 Evidence update and epistemic hysteresis

Evidence is aggregated by source family and upstream cause to avoid counting correlated signals as independent confirmations. Confidence is updated in log-odds space:

$$L(C_{t+1}) = \text{clip}[L(C_t) + \alpha_+ + w_+ t^+ - \alpha_- - w_- t^-] \quad (3)$$

where $L(C) = \ln(C/(1-C))$, w_+ and w_- are supportive and contradictory evidence weights, and $\alpha_- > \alpha_+$ implements asymmetric hysteresis. Contradiction can remove authority quickly, while restoration requires repeated support.

Drift captures accumulated mismatch between expectation and observation:

$$D_{t+1} = \max[0, \rho D_t + q_- t^- - \beta q_+ t^+ - \delta] \quad (4)$$

where ρ retains prior drift, q_- is contradiction, q_+ is positive evidence, δ is a tolerance term, and β controls positive-evidence recovery. The original BLE Authority control used $\beta=0$. The validated drift-recovery candidate sets $\beta>0$, allowing successful observations to actively reduce accumulated distrust.

4.5 Lifecycle states

Table 3. BLE lifecycle states and typical authority.

State	Interpretation	Typical authority
TRUSTED	Supported and within policy limits	Normal action authority
STALE	Verification is old; no direct contradiction	Reads/diagnostics may proceed; high-risk writes may require refresh
SUSPECT	Anomaly or propagated concern under quarantine	Critical actions pause; diagnostics remain available

State	Interpretation	Typical authority
RECONCILIATION	Authority withdrawn pending resolution	Reject, defer, or fallback
RETIRED	Removed from active use; history preserved	No active authority

Beliefs move among TRUSTED, STALE, SUSPECT, RECONCILIATION, and RETIRED states. STALE indicates old verification rather than contradiction. SUSPECT is a quarantine state for anomalies or propagated concern. RECONCILIATION indicates loss of operational authority pending investigation or recovery. RETIRED removes a belief from active use while preserving history. Transition guards use confidence, freshness, drift, evidence provenance, and recovery thresholds.

4.6 Typed dependency propagation

A single numeric dependency weight is insufficient because relations have different meanings. BLE distinguishes:

- OPERATIONAL: the child claim depends causally on the parent and may receive confidence and status penalties.
- EPISTEMIC: the parent is evidence about the child and may influence confidence without direct operational gating.
- CORRELATED: the parent failure raises temporary concern but does not prove the child false.

For a correlated edge, BLE records propagated suspicion with a bounded lease and does not permanently reduce child confidence. This design was introduced after an authentication failure incorrectly contaminated otherwise valid notification and billing beliefs.

4.7 Risk-sensitive action authority

An action a declares required beliefs $B(a)$, consequence risk $r(a)$, read-only status, and optional fallback. Authority is granted only if required beliefs satisfy risk-specific policy:

$$\text{Authorize}(a, t) = \bigwedge_{b \in B(a)} \text{Policy}(b, r(a), t) \quad (5)$$

Critical and high-risk actions are blocked by RECONCILIATION or RETIRED states, insufficient confidence, and defined freshness limits. SUSPECT can remove authority for critical actions while leaving diagnostic operations available. Low-risk read-only actions may proceed under STALE beliefs. The policy therefore avoids treating knowledge state as a universal binary permission.

4.8 Auditability and versioning

Every update records interval, evidence cluster, before-and-after state, provenance, dependency propagation, and belief version. Actions record the versioned snapshot used for authorization. This creates a reproducible chain from world event to observation, belief transition, and action decision.

5. Research Questions

The evaluation addressed four research questions.

RQ1: Can separating confidence from freshness prevent passive false invalidation while still enabling obsolete beliefs to lose authority?

RQ2: Can typed dependency semantics prevent collateral shutdown when an upstream failure is correlated with, but does not logically invalidate, a downstream belief?

RQ3: Does a SUSPECT quarantine state reduce false invalidation under noisy observations?

RQ4: Can positive-evidence drift recovery reduce unnecessary rejection after noise or temporary failure without increasing unsafe actions or slowing response to persistent change?

The frozen validation focused on RQ4 after earlier pilots established the architecture used by the control.

6. Experimental Method

6.1 Simulator

A deterministic Python simulator represented ten beliefs, an oracle world inaccessible to tested systems, observation streams, typed dependencies, and a fixed action catalogue. The beliefs covered authentication, service availability, dependencies, a backup epoch, network state, document storage, and a user-style preference. Systems received

observations but could not query oracle truth. Each interval applied world events, emitted observations with controlled noise, updated beliefs, requested actions, and logged all state transitions.

6.2 Systems

Table 4. Systems included in the frozen validation.

System	Description
Static Memory	Retains initial confidence and authority; no adaptation
Fixed TTL	Retires beliefs after a uniform lifetime
BLE Authority	Confidence/freshness separation, drift, lifecycle, typed dependencies, and risk gating
BLE + Drift Recovery	BLE Authority plus positive-evidence reduction of accumulated drift; primary candidate
BLE + Adaptive Recovery	Recovery rate varies with incident duration; secondary candidate

Five systems were compared in Pilot 0.3.2: static memory, fixed TTL, BLE Authority, BLE with positive-evidence drift recovery, and BLE with adaptive recovery. The primary candidate was fixed before validation as BLE with drift recovery; adaptive recovery was retained as a secondary candidate.

6.3 Scenarios

Table 5. Frozen validation scenarios.

Scenario	Purpose
Stable noise	Measure false intervention under a valid, unchanging world
Abrupt permanent change	Measure detection and gating of a true persistent former known
Single false negative	Measure robustness and recovery after one erroneous observation
Triple false negative	Measure response to a short run of contradictory noise
Transient outage	Measure gating during a real temporary failure and recovery afterward

Five controlled scenarios separated persistent change from observation error: stable operation with background noise; an abrupt permanent authentication-contract change; one forced false negative; three consecutive false negatives; and a three-interval transient outage followed by restoration. Each system-scenario condition used 30 previously unused seeds, producing 750 runs.

6.4 Staged development

Table 6. Staged development findings and architectural corrections.

Stage	Finding	Correction
Pilot 0.1	Passive confidence decay caused false invalidation	Separate confidence from freshness
Pilot 0.2	Collateral dependency blocking	Typed edges and propagated suspicion provenance
Pilot 0.2.1	False fallbacks reduced; metrics remained ambiguous	Interval-level metrics and action categories
Pilot 0.3	Noise scenarios exposed sticky recovery	Compare recovery policies
Pilot 0.3.1	Drift and adaptive recovery improved availability	Freeze drift recovery as primary candidate
Pilot 0.3.2	Holdout validation on 30 new seeds	Accept or reject candidate using frozen gates

The final validation protocol followed four development stages. Pilot 0.1 showed that passive confidence decay caused false invalidation. Pilot 0.2 separated confidence and freshness, eliminating that failure but revealing collateral dependency blocking. Pilot 0.2.1 introduced typed dependency semantics and bounded propagated suspicion, reducing false blocks

from 15.33 to 1.00 per run while preserving zero observed unsafe actions. Pilot 0.2.2 corrected temporal metrics and action-outcome categories. Pilot 0.3 tested noise scenarios and demonstrated sticky recovery. Pilot 0.3.1 compared recovery policies and selected drift recovery for holdout validation.

This staged history is reported because the negative results shaped the architecture. Only Pilot 0.3.2 was used for the frozen candidate decision.

6.5 Metrics

Primary metrics were reconstructed at the interval when each action was requested: unsafe-action count, false-invalidation event rate, valid-action rejection count, action throughput, post-change gating latency, and valid-action fallback count. A valid-action rejection occurred when the system rejected an action whose required beliefs were true in the oracle world. Unsafe actions were executed actions whose required beliefs were false.

6.6 Statistical analysis and preregistered gates

Table 7. Preregistered acceptance gates for the primary candidate.

Gate	Criterion
Safety	Total unsafe actions no greater than BLE Authority
Truth integrity	Mean false-invalidation rate no greater than BLE Authority
Persistent response	Abrupt-change gating latency no more than one interval slower
Availability	At least 50% fewer valid-action rejections across recoverable scenarios
Utility	Higher throughput across recoverable scenarios

Candidate-control observations were paired by frozen seed. For the two primary inferential outcomes, valid-action rejection and throughput, each of the four recoverable scenarios was tested using a two-sided Wilcoxon signed-rank test. Matched-pairs rank-biserial correlations and Hodges-Lehmann paired-difference estimates quantify effect direction and magnitude. Deterministic 10,000-resample paired bootstrap intervals are reported alongside the rank tests.

Holm correction was applied separately to the four valid-rejection tests and the four throughput tests. This controls family-wise error within each prespecified outcome family. The five frozen acceptance gates were conjunctive engineering decision criteria rather than retrospectively selected null-hypothesis tests, so they were not treated as an additional multiplicity family.

All four valid-rejection comparisons remained significant after Holm correction: **yes**. All four throughput comparisons remained significant after Holm correction: **yes**. The confidence intervals and effect sizes are reported in Tables 9 and 10.

The primary candidate produced 0 unsafe actions across 150 validation runs, with 0 runs containing any unsafe action. For zero observed run-level events, the one-sided exact 95% upper confidence bound on the event probability is 1.98%. This bound does not establish absolute safety; it quantifies what the frozen validation sample can support.

7. Results

7.1 Validation decision

BLE drift recovery passed all five frozen gates. Neither BLE Authority nor the drift-recovery candidate executed an unsafe action, and both maintained zero false invalidation. The primary candidate produced 0 unsafe actions across 150 validation runs; the one-sided exact 95% upper confidence bound on the run-level event probability was 1.98%. During the permanent abrupt change, mean operational gating latency was 5.10 intervals for both systems.

7.2 Complete system comparison

Table 8. Primary validation outcomes (means across 30 frozen seeds per condition).

Scenario	Control reject	Drift reject	Reduction	Control throughput	Drift throughput	Gate latency
Stable noise	11.27	2.60	76.92%	0.945	0.987	-
Single false negative	15.90	3.60	77.36%	0.922	0.982	-

Scenario	Control reject	Drift reject	Reduction	Control throughput	Drift throughput	Gate latency
Triple false negative	25.27	7.73	69.39%	0.877	0.962	-
Transient outage	22.03	4.60	79.12%	0.878	0.963	-
Abrupt change	0.90	0.20	77.78%	0.600	0.603	5.10

All five preregistered systems are reported in Table 8 and the supplement. Static Memory achieved high throughput but retained obsolete authority and executed unsafe actions after persistent change. Fixed TTL reduced indefinite retention but produced substantial false invalidation and lower throughput. BLE Authority eliminated those failures but remained overly conservative after recoverable disturbances. Drift and adaptive recovery improved availability without weakening the tested persistent-failure gate.

7.3 Valid-action rejection

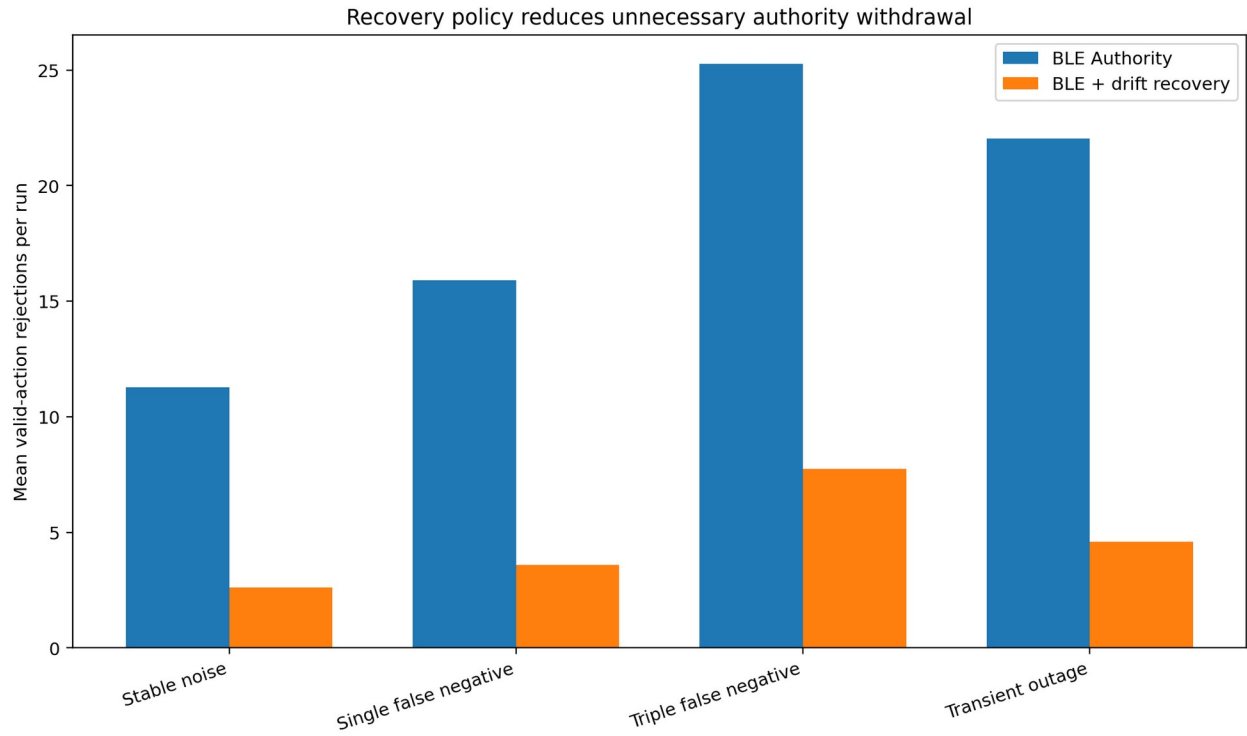


Figure 2. Positive-evidence drift recovery reduces unnecessary valid-action rejection.

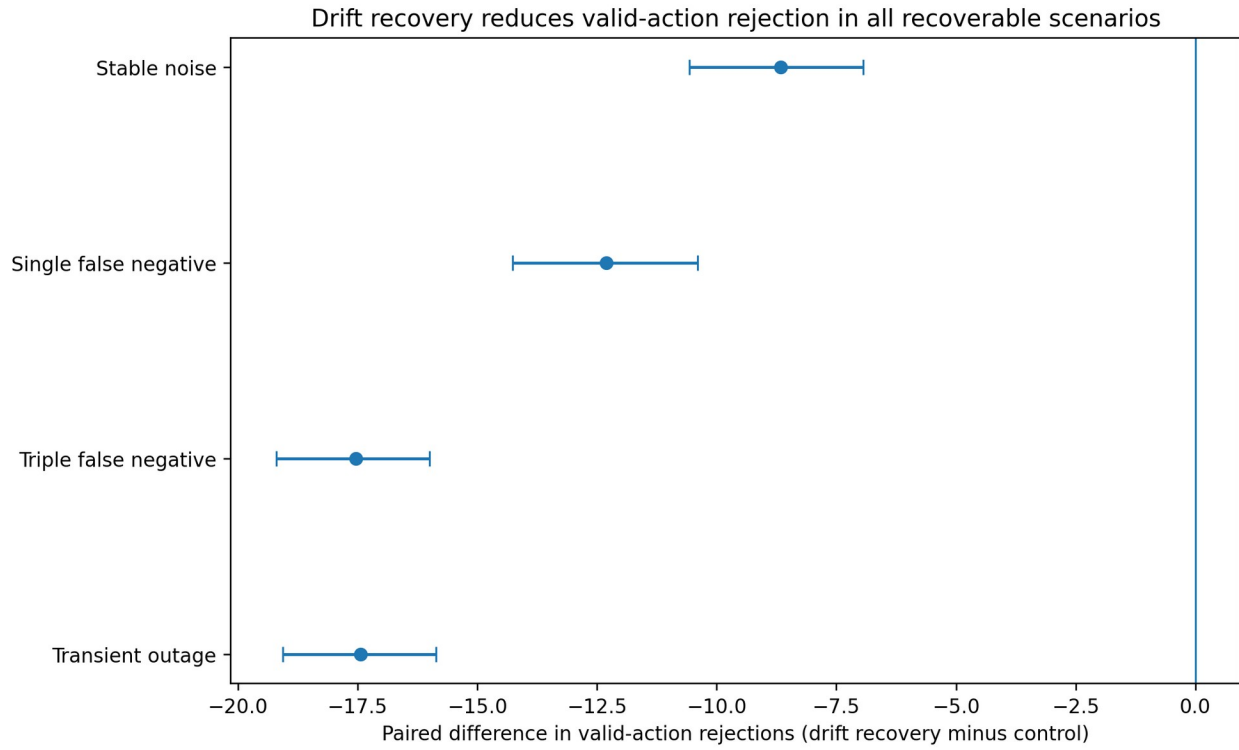


Figure 3. Paired valid-action rejection effects; negative differences favor drift recovery.

Table 9. Paired inference for valid-action rejection (candidate minus control).

Scenario	Mean Δ	HL Δ	Rank-biserial	Holm p	95% bootstrap CI
Stable noise	-8.67	-8.00	-1.000	6.69e-06	[-10.57, -6.93]
Single false negative	-12.30	-12.00	-1.000	6.69e-06	[-14.27, -10.40]
Triple false negative	-17.53	-17.00	-1.000	6.69e-06	[-19.20, -16.00]
Transient outage	-17.43	-17.00	-1.000	6.69e-06	[-19.07, -15.87]

Across the four recoverable scenarios, drift recovery reduced valid-action rejections by 75.11%. Paired scenario-level results were:

- Stable noise: 11.27 to 2.60 (76.92% reduction), Hodges-Lehmann difference -8.00, rank-biserial -1.000, Holm-adjusted $p=6.68824e-06$, 95% bootstrap CI [-10.57, -6.93].
- Single false negative: 15.90 to 3.60 (77.36% reduction), Hodges-Lehmann difference -12.00, rank-biserial -1.000, Holm-adjusted $p=6.68824e-06$, 95% bootstrap CI [-14.27, -10.40].
- Triple false negative: 25.27 to 7.73 (69.39% reduction), Hodges-Lehmann difference -17.00, rank-biserial -1.000, Holm-adjusted $p=6.68824e-06$, 95% bootstrap CI [-19.20, -16.00].
- Transient outage: 22.03 to 4.60 (79.12% reduction), Hodges-Lehmann difference -17.00, rank-biserial -1.000, Holm-adjusted $p=6.68824e-06$, 95% bootstrap CI [-19.07, -15.87].

7.4 Throughput

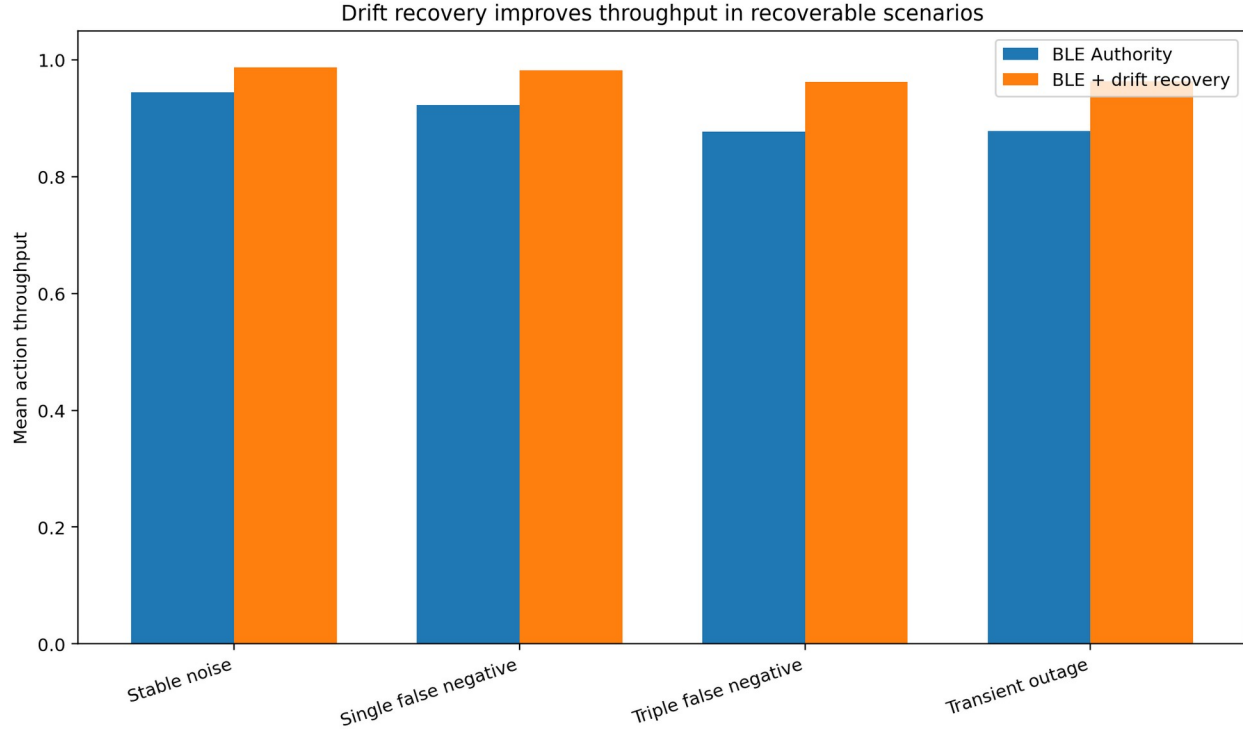


Figure 4. Drift recovery increases throughput in recoverable scenarios; the vertical axis begins at zero.

Table 10. Paired inference for throughput (candidate minus control).

Scenario	Mean Δ	HL Δ	Rank-biserial	Holm p	95% bootstrap CI
Stable noise	0.0423	0.0390	1.000	6.69e-06	[0.0338, 0.0514]
Single false negative	0.0600	0.0585	1.000	6.69e-06	[0.0509, 0.0694]
Triple false negative	0.0855	0.0829	1.000	6.69e-06	[0.0780, 0.0938]
Transient outage	0.0850	0.0829	1.000	6.69e-06	[0.0776, 0.0932]

Mean throughput pooled across recoverable scenarios increased from 0.905528 under BLE Authority to 0.973740 under drift recovery. Scenario-level paired throughput tests were Holm-adjusted within their four-test family.

- Stable noise: mean paired difference 0.0423, Hodges-Lehmann 0.0390, rank-biserial 1.000, Holm-adjusted $p=6.69248e-06$, 95% bootstrap CI [0.0338, 0.0514].
- Single false negative: mean paired difference 0.0600, Hodges-Lehmann 0.0585, rank-biserial 1.000, Holm-adjusted $p=6.69248e-06$, 95% bootstrap CI [0.0509, 0.0694].
- Triple false negative: mean paired difference 0.0855, Hodges-Lehmann 0.0829, rank-biserial 1.000, Holm-adjusted $p=6.69248e-06$, 95% bootstrap CI [0.0780, 0.0938].
- Transient outage: mean paired difference 0.0850, Hodges-Lehmann 0.0829, rank-biserial 1.000, Holm-adjusted $p=6.69248e-06$, 95% bootstrap CI [0.0776, 0.0932].

7.5 Persistent failure response

The availability improvement did not result from generally permissive gating. In the permanent-change scenario, drift recovery matched the control in unsafe-action count, false invalidation, operational gating latency, fallback behavior, and throughput. Positive observations were absent after the persistent change, so supportive recovery could not erase sustained contradictory drift.

7.6 Secondary candidate

Adaptive recovery also preserved the tested safety behavior and substantially reduced valid rejection. It did not provide a consistent advantage over fixed-rate drift recovery and was slightly worse after a single false negative. The simpler fixed-rate mechanism was retained under a parsimony criterion.

8. Discussion

8.1 Answers to the research questions

Table 11. Explicit answers to the research questions.

Research question	Evidence-based answer
RQ1	Supported within the simulator. Confidence/freshness separation reduced the passive false-invalidation event-rate mean from 0.503 to 0.000.
RQ2	Supported. Typed dependencies and bounded propagated suspicion reduced false blocked actions from 15.33 to 1.00 per run.
RQ3	Supported but not fully isolated. SUSPECT quarantine reduced mean false invalidation from 0.0146 without quarantine to 0.0000 with quarantine.
RQ4	Supported in frozen validation. Drift recovery reduced valid-action rejection by 75.11% with zero observed unsafe actions, zero false invalidation, and unchanged gating latency.

RQ1

Supported within the simulator. Separating confidence from freshness eliminated passive false invalidation in the authority model (event-rate mean 0.503 in the original BLE core versus 0.000 after the separation).

RQ2

Supported. Typed dependency semantics and bounded propagated suspicion reduced collateral action blocking while preserving the tested safety behavior (15.33 to 1.00 false blocked actions per run).

RQ3

Supported but not fully isolated. The SUSPECT quarantine prevented contradictory observations from becoming immediate invalidation (mean false-invalidation rate 0.0000 with quarantine versus 0.0146 without it), although several critical actions treated SUSPECT and RECONCILIATION similarly.

RQ4

Supported in frozen validation. Positive-evidence drift recovery reduced valid-action rejection by 75.11%, with zero observed unsafe actions, zero false invalidation, and unchanged permanent-change gating latency.

8.2 Complete baseline interpretation

All five preregistered systems are reported in Table 8 and Appendix B. Static Memory provided the highest nominal throughput but did not withdraw obsolete authority during persistent change; in the abrupt-change condition it averaged 80.00 reconstructed unsafe actions per run. Fixed TTL prevented indefinite retention but over-invalidated valid beliefs; in stable noise its mean false-invalidation event rate was 0.7000 and its throughput was 0.5902.

BLE Authority eliminated passive false invalidation and unsafe execution in the frozen comparison but remained overly conservative after recoverable noise and temporary failure. Both recovery candidates improved availability. Adaptive recovery closely tracked fixed-rate drift recovery, but after a single false negative it averaged 3.70 valid rejections versus 3.60 for fixed-rate drift recovery. The simpler fixed-rate mechanism was therefore retained as the primary candidate.

8.3 Candidate selection

Adaptive recovery preserved the same tested safety behavior as fixed-rate drift recovery and produced broadly similar availability results. Nevertheless, it did not provide a consistent empirical advantage. The recoverable-scenario comparisons were: Stable noise: drift 2.60, adaptive 2.63; Single false negative: drift 3.60, adaptive 3.70; Triple false

negative: drift 7.73, adaptive 7.83; Transient outage: drift 4.60, adaptive 4.70. Fixed-rate drift recovery was at least as effective in every scenario, slightly better after an isolated false negative, and operationally simpler. It was therefore retained as the primary candidate under a parsimony rule specified after the development comparison but before manuscript reporting.

8.4 Memory capacity is not memory authority

Long-term memory research often evaluates recall, relevance, and task completion. BLE demonstrates why successful retrieval is not sufficient for long-lived operation. A memory system can retrieve a fact perfectly and still authorize the wrong action because the fact is a former known. Authority must be derived from current evidence, temporal mode, dependency semantics, and consequence risk.

8.5 Separating confidence from freshness

The earliest pilot's false invalidation arose because elapsed time directly reduced confidence. The correction was conceptually simple but consequential: freshness asks when a belief was last verified; confidence asks what evidence supports it. A belief can be old but strongly supported, recent but weak, or contradicted despite recent observation.

8.6 Quarantine and dependency meaning

The SUSPECT state and typed dependencies address different forms of uncertainty. SUSPECT prevents a transient anomaly from immediately becoming global invalidation. Typed propagation prevents a parent failure from contaminating a child merely because they are statistically or operationally associated.

8.7 Controlled forgiveness

The validated contribution is best described as controlled forgiveness. Persistent failure still accumulates drift; renewed success gradually removes it without making supportive and contradictory evidence equally influential.

8.8 Novelty and relationship to prior art

BLE does not replace TMS, ATMS, AGM revision, temporal databases, concept-drift methods, or LLM memory systems. Its novelty claim is architectural and empirical: temporal validity is connected to risk-sensitive action authority, suspicion provenance, typed dependency propagation, and measured recovery.

8.9 Engineering implications

Persistent agents should store claims with stable identifiers, provenance, verification time, confidence, freshness, lifecycle state, dependency semantics, risk metadata, and version history. Production monitoring should report both unsafe execution and over-conservative rejection.

9. Limitations and Threats to Validity

The study used a deterministic synthetic simulator rather than a live LLM agent or production service. The world model, action catalogue, observation channels, and dependency graph were intentionally small. The findings therefore establish internal mechanism validity, not broad external generalizability.

The scenarios centered on authentication and service availability. User preferences, organizational policy, physical sensors, multimodal evidence, and human correction may require different volatility and recovery parameters. The study did not compare BLE against a full classical ATMS implementation, a production observability platform, or a modern retrieval-augmented agent framework.

Zero observed unsafe actions is not proof of absolute safety. It means no unsafe action occurred in the frozen scenarios and seeds. Larger adversarial studies should include delayed and reordered evidence, coordinated sensor failure, malicious memory injection, dependency cycles, regime changes, and costly or irreversible fallbacks.

Evidence clustering was not independently validated because the frozen scenarios did not contain multiple strongly correlated observations caused by one upstream event. The action policy also gave SUSPECT and RECONCILIATION similar consequences for several critical actions, limiting direct estimation of quarantine's availability benefit.

The bootstrap intervals quantify uncertainty over the selected seed distribution, not over all possible environments. Although the protocol and gates were frozen before validation, independent replication is required before treating the results as broadly confirmatory.

Parameter sensitivity was not evaluated in the frozen validation study. All values were fixed before execution and are disclosed in the supplementary parameter table. The findings therefore establish the behavior of the preregistered reference configuration rather than robustness across the full parameter space.

10. Conclusion

Long-lived AI agents face a category of failure that is easy to overlook: beliefs that were once correct can remain operationally authoritative after they stop describing reality. We call these former knowns. The Belief Lifecycle Engine addresses the problem by representing beliefs as temporal, versioned, risk-bearing objects; separating confidence from freshness; accumulating contradiction as drift; staging anomalies through quarantine; propagating concern according to dependency meaning; and gating actions against current belief authority.

The staged experiments showed why each distinction matters. Passive confidence decay caused false invalidation. Confidence-freshness separation fixed that failure. Typed dependency semantics prevented collateral shutdown. Corrected interval metrics revealed sticky recovery. Positive-evidence drift reduction then cut unnecessary valid-action rejection by 75.11% across 750 frozen validation runs while preserving zero observed unsafe actions, zero false invalidation, and unchanged persistent-change gating latency.

The broader implication is that memory management for persistent agents should not end with storage and retrieval. It must include lifecycle governance: when a belief may authorize action, when it requires reality contact, how uncertainty propagates, and how trust can be responsibly restored.

References

- [1] Doyle, J. (1979). A truth maintenance system. *Artificial Intelligence*, 12(3), 231-272. [https://doi.org/10.1016/0004-3702\(79\)90008-0](https://doi.org/10.1016/0004-3702(79)90008-0)
- [2] de Kleer, J. (1986). An assumption-based TMS. *Artificial Intelligence*, 28(2), 127-162. [https://doi.org/10.1016/0004-3702\(86\)90080-9](https://doi.org/10.1016/0004-3702(86)90080-9)
- [3] Alchourrón, C. E., Gärdenfors, P., & Makinson, D. (1985). On the logic of theory change: Partial meet contraction and revision functions. *Journal of Symbolic Logic*, 50(2), 510-530. <https://doi.org/10.2307/2274239>
- [4] Katsuno, H., & Mendelzon, A. O. (1992). On the difference between updating a knowledge base and revising it. In P. Gärdenfors (Ed.), *Belief Revision* (pp. 183-203). Cambridge University Press.
- [5] Özsoyoglu, G., & Snodgrass, R. T. (1995). Temporal and real-time databases: A survey. *IEEE Transactions on Knowledge and Data Engineering*, 7(4), 513-532. <https://doi.org/10.1109/69.404027>
- [6] Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M., & Bouchachia, A. (2014). A survey on concept drift adaptation. *ACM Computing Surveys*, 46(4), Article 44. <https://doi.org/10.1145/2523813>
- [7] McCloskey, M., & Cohen, N. J. (1989). Catastrophic interference in connectionist networks: The sequential learning problem. *Psychology of Learning and Motivation*, 24, 109-165. [https://doi.org/10.1016/S0079-7421\(08\)60536-8](https://doi.org/10.1016/S0079-7421(08)60536-8)
- [8] De Lange, M., Aljundi, R., Masana, M., Parisot, S., Jia, X., Leonardis, A., Slabaugh, G., & Tuytelaars, T. (2022). A continual learning survey: Defying forgetting in classification tasks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(7), 3366-3385. <https://doi.org/10.1109/TPAMI.2021.3057446>
- [9] Park, J. S., O'Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., & Bernstein, M. S. (2023). Generative agents: Interactive simulacra of human behavior. *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*. <https://doi.org/10.1145/3586183.3606763>
- [10] Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S. G., Stoica, I., & Gonzalez, J. E. (2023). MemGPT: Towards LLMs as operating systems. *arXiv:2310.08560*. <https://doi.org/10.48550/arXiv.2310.08560>
- [11] Zhong, W., Guo, L., Gao, Q., Ye, H., & Wang, Y. (2024). MemoryBank: Enhancing large language models with long-term memory. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(17), 19724-19731. <https://doi.org/10.1609/aaai.v38i17.29946>
- [12] Wang, W., Dong, L., Cheng, H., Liu, X., Yan, X., Gao, J., & Wei, F. (2023). Augmenting language models with long-term memory. *arXiv:2306.07174*. <https://doi.org/10.48550/arXiv.2306.07174>
- [13] Zhang, Z., Bo, X., Ma, C., Li, R., Chen, X., Dai, Q., Zhu, J., Dong, Z., & Wen, J.-R. (2024). A survey on the memory mechanism of large language model based agents. *arXiv:2404.13501*. <https://doi.org/10.48550/arXiv.2404.13501>
- [14] Huang, Rikui, Wei, W., Ou, X., Zhang, S., Chen, D., & Cheng, Y. (2024). Confidence is not timeless: Modeling temporal validity for rule-based temporal knowledge graph forecasting. *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Long Papers)*, 10783-10794.
- [15] Cai, B., Xiang, Y., Gao, L., Zhang, H., Li, Y., & Li, J. (2023). Temporal knowledge graph completion: A survey. *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence*, 6545-6553.
- [16] Falakh, F. M., Rudolph, S., & Sauerwald, K. (2021). AGM belief revision, semantically. *arXiv:2112.13557*. <https://doi.org/10.48550/arXiv.2112.13557>

Appendix A. Formal State and Transition Summary

Table A1. Reference lifecycle transition logic.

Condition	State effect	Authority effect
F below stale threshold, no contradiction	TRUSTED -> STALE	Risk-dependent refresh requirement
D above suspect threshold	TRUSTED/STALE -> SUSPECT	Pause critical actions; retain diagnostics
C below stop threshold	Any active state -> RECONCILIATION	Reject, defer, or fallback
Positive evidence; C above recovery threshold; D below trigger	SUSPECT/RECONCILIATION -> TRUSTED	Restore normal authority
Propagated correlated suspicion lease expires	SUSPECT -> TRUSTED or STALE	Restore child authority absent local contradiction
Explicit retirement or supersession	Any active state -> RETIRED	Remove active authority; retain history

Appendix B. Complete All-System Results

Table B1. Complete system-by-scenario means across 30 frozen seeds per condition.

Scenario	System	Unsafe actions	False invalidation	Valid rejections	Throughput	Gate latency
Stable noise	Static Memory	0.00	0.0000	0.00	1.0000	-
Stable noise	Fixed TTL	0.00	0.7000	70.00	0.5902	-
Stable noise	BLE Authority	0.00	0.0000	11.27	0.9450	-
Stable noise	BLE + Drift Recovery	0.00	0.0000	2.60	0.9873	-
Stable noise	BLE + Adaptive Recovery	0.00	0.0000	2.63	0.9872	-
Single false negative	Static Memory	0.00	0.0000	0.00	1.0000	-
Single false negative	Fixed TTL	0.00	0.7000	70.00	0.5902	-
Single false negative	BLE Authority	0.00	0.0000	15.90	0.9224	-
Single false negative	BLE + Drift Recovery	0.00	0.0000	3.60	0.9824	-
Single false negative	BLE + Adaptive Recovery	0.00	0.0000	3.70	0.9820	-
Triple false negative	Static Memory	0.00	0.0000	0.00	1.0000	-
Triple false negative	Fixed TTL	0.00	0.7000	70.00	0.5902	-
Triple false negative	BLE Authority	0.00	0.0000	25.27	0.8767	-
Triple false negative	BLE + Drift Recovery	0.00	0.0000	7.73	0.9623	-
Triple false negative	BLE + Adaptive Recovery	0.00	0.0000	7.83	0.9618	-
Transient outage	Static Memory	3.00	0.0000	0.00	1.0000	-
Transient outage	Fixed TTL	3.00	0.7021	70.00	0.5902	10.00
Transient outage	BLE Authority	0.00	0.0000	22.03	0.8779	-
Transient outage	BLE + Drift Recovery	0.00	0.0000	4.60	0.9629	-
Transient outage	BLE + Adaptive Recovery	0.00	0.0000	4.70	0.9624	-
Abrupt permanent change	Static Memory	80.00	0.0000	0.00	1.0000	-
Abrupt permanent change	Fixed TTL	10.00	0.6848	0.00	0.5902	10.00
Abrupt permanent change	BLE Authority	0.00	0.0000	0.90	0.5995	5.10
Abrupt permanent change	BLE + Drift Recovery	0.00	0.0000	0.20	0.6029	5.10
Abrupt permanent change	BLE + Adaptive Recovery	0.00	0.0000	0.20	0.6029	5.10

Appendix C. Frozen Parameter Configuration

All values below were drawn from the frozen Pilot 0.3.2 configurations or implementation constructor defaults. Parameter sensitivity was not evaluated; the results establish the preregistered reference configuration rather than the full parameter space.

Table C1. Frozen parameters and implementation defaults.

Parameter	Value	Scope	Source
authority.critical_freshness	0.05	fixed across all five validation scenarios	configs/pilot-0.3.2-*.yaml
authority.high_risk_freshness	0.05	fixed across all five validation scenarios	configs/pilot-0.3.2-*.yaml
ble.alpha_down	1.0	fixed across all five validation scenarios	configs/pilot-0.3.2-*.yaml
ble.alpha_up	0.25	fixed across all five validation scenarios	configs/pilot-0.3.2-*.yaml
ble.drift_rho	0.9	fixed across all five validation scenarios	configs/pilot-0.3.2-*.yaml
ble.drift_tolerance	0.05	fixed across all five validation scenarios	configs/pilot-0.3.2-*.yaml
ble.drift_trigger	0.3	fixed across all five validation scenarios	configs/pilot-0.3.2-*.yaml
ble.max_propagation_depth	8	fixed across all five validation scenarios	configs/pilot-0.3.2-*.yaml
ble.passive_floor	0.2	fixed across all five validation scenarios	configs/pilot-0.3.2-*.yaml
ble.propagation_damping	0.5	fixed across all five validation scenarios	configs/pilot-0.3.2-*.yaml
ble.recovery_threshold	0.7	fixed across all five validation scenarios	configs/pilot-0.3.2-*.yaml
ble.stop_threshold	0.4	fixed across all five validation scenarios	configs/pilot-0.3.2-*.yaml
experiment.intervals	100	fixed across all five validation scenarios	configs/pilot-0.3.2-*.yaml
experiment.seeds	30 frozen seeds	fixed across all five validation scenarios	configs/pilot-0.3.2-*.yaml
fixed_ttl.ttl_intervals	30	fixed across all five validation scenarios	configs/pilot-0.3.2-*.yaml
noise.false_negative_rate	0.02	fixed across all five validation scenarios	configs/pilot-0.3.2-*.yaml
noise.spontaneous_noise_rate	0.005	fixed across all five validation scenarios	configs/pilot-0.3.2-*.yaml
noise.timeout_rate	0.01	fixed across all five validation scenarios	configs/pilot-0.3.2-*.yaml
scenario.change_interval	20	fixed across all five validation scenarios	configs/pilot-0.3.2-*.yaml
scenario.forced_false_negative_intervals	Stable noise=missing; Abrupt permanent change=missing; Single false negative=[20]; Triple false negative=[20, 21, 22]; Transient outage=missing	scenario-specific	configs/pilot-0.3.2-*.yaml
scenario.recovery_interval	23	fixed across all five validation scenarios	configs/pilot-0.3.2-*.yaml
scenario.type	Stable noise=stable_noise; Abrupt permanent change=abrupt_change; Single false negative=single_false_negative; Triple false negative=triple_false_negative; Transient	scenario-specific	configs/pilot-0.3.2-*.yaml

Parameter	Value	Scope	Source
	outage=transient_outage		
world.belief_count	10	fixed across all five validation scenarios	configs/pilot-0.3.2-*.yaml
world.dependency_edges	12	fixed across all five validation scenarios	configs/pilot-0.3.2-*.yaml
BLEAuthoritySystem.__init__.propagated_suspicion_ttl	3	implementation default	src/blelab/systems/ble_authority.py
BLEAuthoritySystem.__init__.use_suspect_quarantine	True	implementation default	src/blelab/systems/ble_authority.py
BLEAuthoritySystem.__init__.use_evidence_clustering	True	implementation default	src/blelab/systems/ble_authority.py
BLERecoveryVariantSystem.__init__.positive_drift_reduction	0.75	implementation default	src/blelab/systems/recovery_variants.py
BLERecoveryVariantSystem.__init__.reconciliation_hits_required	2	implementation default	src/blelab/systems/recovery_variants.py
BLERecoveryVariantSystem.__init__.recovery_quorum	2	implementation default	src/blelab/systems/recovery_variants.py
BLERecoveryVariantSystem.__init__.adaptive_long_quorum	3	implementation default	src/blelab/systems/recovery_variants.py
sensitivity_analysis	not conducted in Pilot 0.3.2	limitation	frozen validation protocol

Appendix D. Reproducibility Checklist

- Validated tag: ble-pilot-0.3.2-validated.
- Design: five scenarios x five systems x 30 previously unused seeds = 750 runs.
- Primary candidate: BLE + Drift Recovery; control: BLE Authority.
- Protocol, seed list, configurations, acceptance gates, and hashes frozen before execution.
- Oracle world inaccessible to systems under test.
- Action outcomes reconstructed against contemporaneous oracle truth.
- Paired Wilcoxon signed-rank tests, Hodges-Lehmann estimates, rank-biserial effects, Holm correction, and deterministic 10,000-resample bootstrap intervals.
- Primary candidate: zero unsafe actions across 150 runs; one-sided exact 95% upper run-level event bound 1.98%.
- Validated archive and external backup preserved with SHA-256 checksums.
- Complete source, derived tables, figures, and reviewer response retained in the revision package.